

Saksham Gupta

Systems & Infrastructure Engineer | Performance Engineering, Framework Architecture, Polyglot Runtimes

✉ sakshamdham54321@gmail.com ☎ +91 8687-667561 📝 Blog in sakshamgupta28 🌀 Omega28

Experience

Oracle, Project Lead [Accelerated Promotion]

Bengaluru, IN Oct 2025 – Present

- Architected a **high-performance declarative rendering engine** (Siebel Web Component Framework) with a recursive JSON layout grammar, composable expression layer, and signal-based reactive state management using Preact internals — efficiently handling large DOM trees with strong validation and debuggability.
- Designed a **DSL-driven integration orchestration framework** (Siebel Open Integration – Phase 2) with external DSL + internal DSL model for composing multi-step workflows across REST, eventing, and data-access layers with declarative transitions, conditions, and state management.
- Established **CI/CD pipelines** and built an AI-assisted unit testing framework, authoring **100+ test cases** to enforce code quality standards across the team.
- Mentored engineers across 3 stacks (Java, JavaScript, Python), led architecture reviews, and authored comprehensive design documents to drive design-first engineering culture.

Oracle, Senior Software Engineer

Bengaluru, IN Sept 2024 – Sept 2025

- Designed and implemented a **RAG (Retrieval-Augmented Generation) system** for Siebel Bookshelf, integrating vector databases, LLMs, and LangChain for intelligent document retrieval; led a team of two engineers through delivery.
- Spearheaded migration of **Siebel Open Integration from Java 17 to Java 21** with Virtual Threads and Helidon 4 upgrade, resolving critical synchronous/asynchronous proxy bugs and authoring a new error-handling module.
- Automated third-party dependency management by implementing a **declarative package.json-based system**, eliminating manual downloads — awarded “**Continuous Innovation**” Award.
- **Integrated a modern component framework** (Preact + Web Components) into Siebel OpenUI, handling bundling, module loading, VDOM lifecycle, and component architecture.
- Explored and evaluated **OpenTelemetry** for distributed tracing and observability across microservices.

Oracle, Software Engineer

Bengaluru, IN Aug 2022 – Aug 2024

- Led **Siebel Open Integration from proof-of-concept to beta in 12 months**, building the entire server-side execution layer from scratch.
- **Reduced CPU usage by 60% and API latency by 75%** through systematic profiling with flamegraphs and CPU samplers, optimizing hot code paths, session management, and resource preemption logic.
- Engineered a **polyglot runtime** using GraalVM JavaScript contexts embedded in a Java Helidon service, enabling server-side execution of client-side business logic in a multi-tenant environment.
- Built **automated REST API code generation** using OpenAPI specifications and GraalVM polyglot capabilities, exposing Siebel domain objects as typed resource hierarchies with a developer-friendly DSL.
- Reached **top 6 out of 101 teams** in a global Generative AI hackathon.

Skills

Languages: Java, C++, JavaScript, Bash, Python, Rust, Haskell

Systems & Infrastructure: GraalVM Polyglot, Helidon, Virtual Threads, performance profiling (flamegraphs, CPU samplers, VisualVM), concurrency & multithreading, REST API design, code generation (OpenAPI)

Architecture & Design: DSL design (external/internal), workflow orchestration, framework architecture, reactive programming, recursive descent parsing

AI/ML & Observability: RAG pipelines, LangChain, vector search / embeddings, OpenTelemetry, distributed tracing

Frontend: Preact, React, Web Components, signal-based state management, JSON-driven UI rendering

Education

Indian Institute of Technology, Roorkee BTech, Computer Science

2018 – 2022

- **GPA:** 8.42 ([Degree](#)) | **JEE (Advanced):** AIR 353 ([Result](#))
- **Capstone:** ([GitHub](#)) Built a CPU Simulator in C++ simulating Out-of-Order (O3) execution with Scoreboarding and branch prediction on a RISC-V-inspired ISA.

Awards

- **2x “Continuous Innovation” Award** (FY24 Q4, FY25 Q3) — recognized for building a greenfield API platform with GraalVM polyglot and automating dependency management.
- **“Agency in Action” Award** (FY25 Q3) — resolved 25+ compliance and accessibility issues, directly supporting critical release deliveries.
- **4 consecutive “Outstanding” performance ratings** (FY23–FY26) with accelerated promotion from IC2 to Project Lead.
- **Top 6 in Oracle Global GenAI Hackathon** — 101 teams worldwide.

Projects

- **OS From Scratch** (C, x86 Assembly) ([GitHub](#)) — 32-bit UNIX clone kernel with custom bootloader, interrupt handling, virtual memory manager, VGA driver, memory allocator, and shell. Multi-boot compliant; runs on QEMU.
- **CPU Simulator — CENOS** (C++) ([GitHub](#)) — Simulates Out-of-Order execution pipeline with Scoreboarding, branch prediction, and a 2-pass assembler on a RISC-V-inspired ISA.
- **HTTP Proxy Server** (C++) ([GitHub](#)) — Minimal HTTP proxy using RAI for socket safety and custom URL parsing; forwards client requests to remote hosts.
- **SAVY — Synchronized Video Player** (JavaScript) ([GitHub](#)) — Real-time video synchronization across clients using Node.js, Socket.io, and WebRTC audio chat.
- **CacheCraft** (Java) ([GitHub](#)) — Implementation of various caching strategies and design patterns (LRU, LFU, write-through, write-back).
- **JSON Parser** (Java) ([GitHub](#)) — Recursive descent parser for JSON objects, arrays, strings, numbers, booleans, and null values.
- **Math Expression Evaluator** (Java) ([GitHub](#)) — Expression parser & evaluator using functional parser-combinator techniques.

Technical Writing

- **Blog:** 0mega28.github.io — Deep dives into Java internals, distributed systems, and systems programming.
- **A Systems-First Deep Dive into the Spark Paper** — Dissecting the 2012 RDD paper: how Spark’s functional, lineage-based architecture solved fault tolerance without replication.
- **The JVM Has Never Heard of Your List<String>** — Technical deep dive into Java generics type erasure and bytecode-level implications.
- **The Halting Problem** — How Turing proved no program can predict whether arbitrary code will halt, using self-reference and contradiction.